Smart Exam Cheating Detection System

Prepared By:

S.No.	Name	Roll Number	Role
1	**Naitik Kumar**	2410030859	**Group Leader**
2	Vijendra Kumar	2410030862	Member
3	Sunny Kumar	2410030829	Member
4	Ashish Kumar Singh	25SCS1003004431	Member

Group Leader: NAITIK KUMAR

Submitted to: MR. ADITYA BANGIA

Department: COMPUTER SCIENCE

Subject: DBMS

1. Project Overview

The Smart Exam Cheating Detection System is a MariaDB/MySQL-based project that monitors and analyses student behaviour during online exams using SQL queries. It detects suspicious patterns such as shared IP addresses, identical answer submissions, excessive browser tab switching, and unusual submission timings — all without using artificial intelligence.

This document demonstrates the normalization of the database schema from an Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), and finally Third Normal Form (3NF).

2. Initial Database Schema (Before Normalization)

To demonstrate normalization, we first imagine all the data combined into a single flat unnormalized table called EXAM_SYSTEM. This represents the worst-case starting point — all columns in one place with redundant data.

Column	Sample Value	Problem
student_id	1	Repeated for each exam attempt
student_name	Arjun Sharma	Repeated for each exam attempt
roll_no	CS2101	Repeated for each exam attempt
email	arjun@college.edu	Repeated for each exam attempt
department	CSE	Repeated for each exam attempt
exam_id	1	Repeated for each question answered
subject	Database Mgmt Systems	Repeated for each question answered
exam_date	2025-03-10	Repeated for each question answered
duration	120	Repeated for each question answered
total_marks	100	Repeated for each question answered
question_id	1	Repeated for each student attempt
question_text	What does SQL stand for?	Repeated for each student attempt
correct_option	C	Repeated for each student attempt
attempt_id	1	Core tracking unit
start_time	2025-03-10 09:00:00	Repeated for each question answered
end_time	2025-03-10 10:55:00	Repeated for each question answered
ip_address	192.168.1.10	Repeated for each question answered
device_id	DEV001	Repeated for each question answered
total_tab_switches	2	Repeated for each question answered

cheating_detected	FALSE	Repeated for each question answered
selected_option	C	One per student per question
submission_time	2025-03-10 09:10:00	One per student per question
flag_id	1	Multiple flags per attempt
reason	Shared IP address...	Multiple flags per attempt
severity_level	4	Multiple flags per attempt
detected_at	2025-03-10 09:00:00	Multiple flags per attempt

3 Functional Dependencies (FDs)

A Functional Dependency (FD) X → Y means: knowing the value of X uniquely determines the value of Y. Below are all FDs identified in our schema:

Determinant (X)	Determines → Dependent Attributes (Y)
student_id	student_name, roll_no, email, department
roll_no	student_id, student_name, email, department
exam_id	subject, exam_date, duration, total_marks
question_id	question_text, correct_option, exam_id
attempt_id	student_id, exam_id, start_time, end_time, ip_address, device_id, total_tab_switches, cheating_detected
(attempt_id, question_id)	selected_option, submission_time
flag_id	attempt_id, reason, severity_level, detected_at

Key Observations:

• **student_id → department** but **department** does NOT determine student_id (many students department). So department is non-key, fully dependent on student_id.

• **(attempt_id, question_id)** is a composite key for the ANSWER data. selected_option and submission_time depend on BOTH — not just one.

• **attempt_id → student_id → department** is a transitive dependency (department depends on student_id, which itself depends on attempt_id).

4 Step 1 — First Normal Form (1NF)

A table is in **1NF** if: (1) Each column contains only **atomic (indivisible) values**. (2) There are no **repeating groups** or arrays. (3) Each row is **uniquely identifiable** by a primary key.

Violations in UNF table

The unnormalized **EXAM_SYSTEM** table has **repeating groups** — for one exam attempt, multiple questions and multiple flags create multi-valued rows. For example, attempt_id=8 has 2 cheating flags (severity 5 each) creating duplicate rows for the same attempt.

1NF Solution — Ensure one value per cell, unique primary keys

In 1NF we keep all columns but ensure no repeating groups. Each row gets a composite primary key: **(attempt_id, question_id, flag_id)**. Every cell now has exactly one value.

EXAM_SYSTEM table after 1NF:

Column	Type	Note
attempt_id *	INT	Part of composite PK
question_id *	INT	Part of composite PK
flag_id *	INT	Part of composite PK
student_id	INT	Atomic value
student_name	VARCHAR	Atomic value
roll_no	VARCHAR	Atomic value
email	VARCHAR	Atomic value
department	VARCHAR	Atomic value

exam_id	INT	Atomic value
subject	VARCHAR	Atomic value
exam_date	DATE	Atomic value
duration	INT	Atomic value
total_marks	INT	Atomic value
question_text	VARCHAR	Atomic value
correct_option	CHAR(1)	Atomic value
start_time	DATETIME	Atomic value
end_time	DATETIME	Atomic value
ip_address	VARCHAR	Atomic value
device_id	VARCHAR	Atomic value
total_tab_switches	INT	Atomic value
cheating_detected	BOOLEAN	Atomic value
selected_option	CHAR(1)	Atomic value
submission_time	DATETIME	Atomic value
reason	VARCHAR	Atomic value
severity_level	INT	Atomic value
detected_at	DATETIME	Atomic value

1NF Achieved — All values are atomic. Each row is uniquely identified by **(attempt_id, question_id, flag_id)**. No arrays or multi-valued cells exist. However, massive redundancy remains — student_name repeats for every question every student answers.

5. Step 2 — Second Normal Form (2NF)

A table is in **2NF** if: (1) It is already in 1NF. (2) Every non-key attribute is **fully functionally dependent** on the entire primary key (no partial dependencies).

Partial Dependencies Found in 1NF Table

The composite PK is **(attempt_id, question_id, flag_id).** Many attributes depend on only part of this key these are partial dependencies:

Attribute	Depends On	Partial Dependency? (Violation)
student_name, roll_no, email, department	student_id only	YES — violates 2NF
subject, exam_date, duration, total_marks	exam_id only	YES — violates 2NF
question_text, correct_option	question_id only	YES — violates 2NF
start_time, end_time, ip_address, device_id, total_tab_switches, cheating_detected	attempt_id only	YES — violates 2NF
selected_option, submission_time	(attempt_id, question_id)	NO — fully dependent
reason, severity_level, detected_at	flag_id only	YES — violates 2NF

2NF Solution — Decompose into Separate Tables

Each group of attributes that depends on only part of the composite key is extracted into its own table. This eliminates all partial dependencies.

Table 1: STUDENT (student_id → all student attributes)

Column	Type	Constraint
student_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
roll_no	VARCHAR(20)	UNIQUE, NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
department	VARCHAR(50)	NOT NULL

Table 2: EXAM (exam_id → all exam attributes)

Column	Type	Constraint
exam_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
subject	VARCHAR(100)	NOT NULL
exam_date	DATE	NOT NULL
duration	INT	NOT NULL
total_marks	INT	NOT NULL

Table 3: QUESTION (question_id → question text and correct option)

Column	Type	Constraint
question_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
exam_id (FK)	INT	REFERENCES EXAM(exam_id)
question_text	VARCHAR(500)	NOT NULL
correct_option	CHAR(1)	NOT NULL

Table 4: STUDENT_EXAM (attempt_id → attempt details)

Column	Type	Constraint
attempt_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
student_id (FK)	INT	REFERENCES STUDENT(student_id)
exam_id (FK)	INT	REFERENCES EXAM(exam_id)
start_time	DATETIME	NOT NULL
end_time	DATETIME	NULL
ip_address	VARCHAR(45)	NOT NULL
device_id	VARCHAR(100)	NOT NULL
total_tab_switches	INT	DEFAULT 0
cheating_detected	BOOLEAN	DEFAULT FALSE

Table 5: ANSWER ((attempt_id, question_id) → selected answer)

Column	Type	Constraint
answer_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
attempt_id (FK)	INT	REFERENCES STUDENT_EXAM(attempt_id)
question_id (FK)	INT	REFERENCES QUESTION(question_id)
selected_option	CHAR(1)	NOT NULL
submission_time	DATETIME	NOT NULL

Table 6: CHEATING_FLAG (flag_id → flag details)

Column	Type	Constraint
flag_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
attempt_id (FK)	INT	REFERENCES STUDENT_EXAM(attempt_id)
reason	VARCHAR(255)	NOT NULL
severity_level	INT	CHECK (1–5)
detected_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

2NF Achieved — All partial dependencies eliminated. Every non-key attribute now depends on the complete primary key of its own table. No attribute depends on only a subset of its table's primary key.

6 Step 3 — Third Normal Form (3NF)

A table is in 3NF if: (1) It is already in 2NF. (2) There are no transitive dependencies — i.e., no non-key attribute determines another non-key attribute.

Checking for Transitive Dependencies in 2NF Tables

A transitive dependency exists when: PK → A → B (B depends on A, which depends on PK, but B is not directly dependent on PK). We check each 2NF table:

Table	Possible Transitive Dependency	Violation?
STUDENT	student_id → department → dept_hod (if stored)But we only store department name — no further dependency	NO
EXAM	exam_id → subject → subject_credits (if stored)We only store subject name — no further dependency	NO
QUESTION	question_id → exam_id (FK, not transitive)correct_option depends on question_id directly	NO
STUDENT_EXAM	attempt_id → student_id → name/departmentBut name/department are in STUDENT table	NO (resolved by 2NF)
ANSWER	(attempt_id, question_id) → selected_option, submission_timeNo non-key determines another non-key	NO
CHEATING_FLAG	flag_id → attempt_id (FK)reason, severity_level depend directly on flag_id	NO

Result: After 2NF decomposition, NO transitive dependencies remain in any table. The schema is already in 3NF. This is because the 2NF decomposition correctly separated all entity groups student data, exam data, question data, attempt data, answers, and flags each into their own table with a clear single primary key.

Why No Transitive Dependencies Remain

In the original flat table, attempt_id → student_id → department was a transitive dependency (department was transitively dependent on attempt_id via student_id). This was eliminated in 2NF when we created the STUDENT table. Now:

- STUDENT_EXAM only holds: attempt_id, student_id (FK), exam_id (FK), ip_address, device_id, tab_switches, cheating_detected.
- Student name and department live in STUDENT table where student_id is the PK.
- No non-key column in any table determines another non-key column in the same table.

7. Final Normalized Schema (3NF)

The final normalized database consists of 6 tables, each in 3NF. Below is the complete schema with all constraints:

Table: STUDENT

Column	Data Type	Constraint
student_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
roll_no	VARCHAR(20)	UNIQUE, NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
department	VARCHAR(50)	NOT NULL

Table: EXAM

Column	Data Type	Constraint
exam_id	INT	PK, AUTO_INCREMENT
subject	VARCHAR(100)	NOT NULL
exam_date	DATE	NOT NULL
duration	INT	NOT NULL (minutes)
total_marks	INT	NOT NULL

Table: QUESTION

Column	Data Type	Constraint
question_id	INT	PK, AUTO_INCREMENT
exam_id	INT	FK → EXAM(exam_id)
question_text	VARCHAR(500)	NOT NULL
correct_option	CHAR(1)	NOT NULL

Table: STUDENT_EXAM

Column	Data Type	Constraint
attempt_id	INT	PK, AUTO_INCREMENT
student_id	INT	FK → STUDENT(student_id)
exam_id	INT	FK → EXAM(exam_id)
start_time	DATETIME	NOT NULL
end_time	DATETIME	NULL
ip_address	VARCHAR(45)	NOT NULL
device_id	VARCHAR(100)	NOT NULL
total_tab_switches	INT	DEFAULT 0
cheating_detected	BOOLEAN	DEFAULT FALSE

Table: ANSWER

Column	Data Type	Constraint
answer_id	INT	PK, AUTO_INCREMENT
attempt_id	INT	FK → STUDENT_EXAM(attempt_id)
question_id	INT	FK → QUESTION(question_id)
selected_option	CHAR(1)	NOT NULL
submission_time	DATETIME	NOT NULL

Table: CHEATING_FLAG

Column	Data Type	Constraint
flag_id	INT	PK, AUTO_INCREMENT
attempt_id	INT	FK → STUDENT_EXAM(attempt_id)
reason	VARCHAR(255)	NOT NULL
severity_level	INT	CHECK (1 to 5)
detected_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

8. Normalization Summary

At the beginning, all the data was stored in one single table, which caused a lot of repetition and confusion. The same student, exam, and question details were repeated again and again, making the database messy.

In the first step (1NF), all values were made atomic and repeating groups were removed. Each record became unique with the help of a primary key, but redundancy was still there.

In the second step (2NF), partial dependencies were removed by dividing the big table into smaller tables like STUDENT, EXAM, QUESTION, etc. This reduced duplication and made the data more organized.

Finally, in the third step (3NF), transitive dependencies were removed. Now, all the tables are properly structured and each field depends only on its primary key. The database is now clean and efficient.

9. Benefits of Normalization

- Reduces data repetition and keeps the database clean
- Makes updating data easier and faster
- Prevents data loss while deleting records
- Allows easy insertion of new data
- Maintains proper relationships between tables

9. Conclusion

The database for the Smart Exam Cheating Detection System has been successfully normalized up to Third Normal Form (3NF). Initially, all the data was stored in a single table, which caused a lot of redundancy and made the system inefficient.

By applying normalization step by step (1NF, 2NF, and 3NF), the database was divided into smaller and well-structured tables. This removed unnecessary repetition and made the data easier to manage.

The final schema consists of 6 tables with proper relationships using foreign keys. It avoids update, insertion, and deletion problems, and keeps the data consistent.